

EasyFood - Missão Sprint 03

Persistência, teste da arquitetura e registro de decisões

Na última aula, evoluímos nossa API. A EasyFood agora consegue consultar restaurantes e cadastrar novos restaurantes.

Agora chegou o momento de testar uma limitação importante da nossa arquitetura e registrar uma das decisões tomadas até aqui.

```
GET /restaurants
POST /restaurants
```

Parte 1 - PREPARE

Deixe sua EasyFood no mesmo ponto da aula

Antes de começar a missão, sua aplicação deve possuir as duas rotas abaixo:

```
GET /restaurants
POST /restaurants
```

Abra o terminal dentro do projeto e execute:

```
node server.js
```

Você deverá visualizar:

```
EasyFood rodando na porta 3000
```

1. Teste o GET

```
GET http://localhost:3000/restaurants
```

Você deverá receber a lista de restaurantes cadastrados.

```
[
  {
    "id": 1,
    "name": "Pizzaria Napoli",
    "category": "Pizza",
    "rating": 4.8
  },
  {
    "id": 2,
    "name": "Burger House",
    "category": "Hambúrguer",
    "rating": 4.7
  },
  {
    "id": 3,
    "name": "Sushi House",
    "category": "Japonês",
    "rating": 4.9
  }
]
```

Se funcionou: GET /restaurants - OK.

2. Teste o POST

No Postman, faça uma requisição POST para:

```
POST http://localhost:3000/restaurants
```

No Body, selecione raw e JSON. Envie:

```
{
  "name": "Taco Loco",
  "category": "Mexicana",
  "rating": 4.3
}
```

A resposta deverá ser semelhante a:

```
{
  "id": 4,
  "name": "Taco Loco",
  "category": "Mexicana",
  "rating": 4.3
}
```

Status esperado: 201 Created.

Se funcionou: POST /restaurants - OK.

3. Confirme o cadastro

Faça novamente:

```
GET http://localhost:3000/restaurants
```

O novo restaurante deverá aparecer na lista.

EASYFOOD API

GET /restaurants	POST /restaurants
CONSULTA	CADASTRA
+----- ARRAY -----+	

Até aqui, tudo parece funcionar. Mas será que nossos dados estão realmente salvos?

Parte 2 - TESTE

4. Cadastre um novo restaurante

Faça novamente um POST /restaurants e cadastre qualquer restaurante que desejar. Exemplo:

```
{
  "name": "Cantina Roma",
  "category": "Italiana",
  "rating": 4.5
}
```

Depois faça GET /restaurants e confirme que o restaurante apareceu.

5. Reinicie o servidor

No terminal, pare a aplicação:

```
Ctrl + C
```

Agora inicie novamente:

```
node server.js
```

Faça outra vez:

```
GET /restaurants
```

O que aconteceu?

O restaurante cadastrado anteriormente desapareceu.

Isso acontece porque estamos armazenando os restaurantes em um array na memória da aplicação:

```
const restaurants = [
  {
    id: 1,
    name: "Pizzaria Napoli",
    category: "Pizza",
    rating: 4.8
  }
];
```

Quando cadastramos um novo restaurante, utilizamos:

```
restaurants.push(novoRestaurante);
```

Enquanto o processo Node.js estiver executando, os dados existem. Quando o processo é encerrado, a memória é perdida.

```
Servidor encerra
|
Memória é perdida
|
Dados cadastrados desaparecem
```

Isso não é necessariamente um erro. Foi uma decisão arquitetural que permitiu desenvolver e testar rapidamente nossa primeira versão. Decisões arquiteturais possuem consequências.

Parte 3 - REGISTRE A DECISÃO

Durante o desenvolvimento da EasyFood tomamos uma decisão: nesta primeira versão, os restaurantes serão armazenados em memória.

Agora precisamos documentá-la utilizando um ADR - Architecture Decision Record.

Um ADR registra uma decisão arquitetural importante do projeto e explica o contexto, as alternativas, a decisão, a justificativa e suas consequências.

6. Crie a pasta de ADRs

```
easy-food/
|
+-- docs/
|   +-- adr/
|   |
+-- node_modules/
+-- package.json
+-- package-lock.json
+-- server.js
```

7. Crie nosso primeiro ADR

Dentro de docs/adr/, crie:

```
ADR-001-armazenar-restaurantes-em-memoria.md
```

A extensão .md significa Markdown, um formato de texto muito utilizado para documentação de projetos de software.

Conteúdo do ADR-001

```
# ADR-001 - Armazenar restaurantes em memória

## Status
Aceita
```

Data
20/08/2026

Responsável
Equipe EasyFood

Contexto
Estamos desenvolvendo a primeira versão da API da EasyFood.

Neste momento, a aplicação precisa permitir:

- consultar restaurantes;
- cadastrar novos restaurantes.

O produto ainda está em fase de prototipação, teste e validação.
A prioridade desta primeira versão é validar o fluxo da aplicação de forma rápida e simples antes de aumentar a complexidade da arquitetura.

Alternativas consideradas

1. Array em memória
2. PostgreSQL
3. MongoDB
4. SQLite
5. Firebase
6. Arquivo JSON

Decisão

Adotar um array em memória como mecanismo de armazenamento dos restaurantes na versão inicial do serviço.

Justificativa

- Permite maior velocidade no desenvolvimento.
- Facilita os primeiros testes da API.
- Possui baixa complexidade.
- Não exige configuração de infraestrutura.
- Não possui custo adicional para esta fase do projeto.

Consequências

Positivas

- Desenvolvimento mais rápido.
- Facilidade para testar GET e POST.
- Menor complexidade inicial.
- Permite validar o conceito da aplicação rapidamente.

Negativas

- Os dados são perdidos quando o servidor reinicia.
- Não existe persistência dos dados.
- Não é adequado para múltiplas instâncias da aplicação.
- Possui limitações para consultas e análises mais complexas.
- Não oferece os mesmos mecanismos de integridade disponíveis em um banco de dados.

Critérios de revisão

Esta decisão deverá ser reavaliada quando:

1. O MVP for validado e houver decisão de avançar para produção.
2. Houver necessidade de manter os dados entre reinicializações e deploys.
3. O volume de dados ultrapassar o que é razoável manter em memória.
4. For necessário realizar consultas mais complexas.
5. Surgirem relacionamentos entre diferentes entidades da aplicação.

Notas

Esta é uma decisão temporária para a fase inicial da EasyFood.

Antes da entrada em produção, deverá ser avaliado um mecanismo adequado de persistência.

A escolha desse mecanismo deverá gerar uma nova decisão arquitetural:
ADR-002 - Escolha do mecanismo de persistência da EasyFood.

O que acabamos de fazer?

```
DECISÃO
|
CÓDIGO
|
TESTE
|
CONSEQUÊNCIA
|
DOCUMENTAÇÃO
```

Agora nosso projeto possui não apenas código, mas também o registro das decisões que levaram esse código a existir.

Parte 4 - EVOLUA

Já descobrimos que armazenar restaurantes em memória possui uma limitação importante: os dados desaparecem quando o servidor reinicia.

Agora precisamos evoluir nossa arquitetura.

Como conectaríamos nossa API a um banco de dados?

Você pode considerar PostgreSQL, MongoDB, SQLite, Firebase ou outra tecnologia que considere adequada.

Sua missão

- [] Qual banco de dados você escolheria para a EasyFood?
- [] Por que escolheria essa tecnologia?
- [] O que precisaria mudar no server.js?
- [] Onde o restaurante seria salvo?
- [] Como o GET /restaurants buscaria esses dados?
- [] Como o POST /restaurants salvaria esses dados?
- [] Precisaríamos instalar alguma nova dependência?
- [] Nossa arquitetura precisaria mudar?
- [] O desenho arquitetural da EasyFood precisaria mudar?
- [] Quais vantagens essa decisão traria?
- [] Quais novos problemas ou trade-offs ela poderia criar?

Desafio opcional

Tente conectar a EasyFood a um banco de dados. Você pode escolher a tecnologia.

O objetivo não é encontrar a implementação perfeita. O objetivo é experimentar.

```
POST /restaurants
|
Banco de dados
|
GET /restaurants
```

Depois, cadastre um restaurante, reinicie o servidor e faça novamente o GET. Se o restaurante continuar lá, você resolveu o problema de persistência.

Desafio de arquitetura

Se você implementar uma solução com banco de dados, não entregue apenas o código.

Crie também:

```
docs/adr/ADR-002-escolha-do-banco-de-dados.md
```

Registre: Contexto, Alternativas consideradas, Decisão, Justificativa, Consequências positivas, Consequências negativas e Critérios de revisão.

Um arquiteto não deve apenas dizer qual tecnologia escolheu. Deve conseguir explicar por que escolheu.

Para a próxima aula

- [] GET /restaurants funcionando
- [] POST /restaurants funcionando
- [] ADR-001-armazenar-restaurantes-em-memoria.md criado
- [] Teste de reinicialização realizado
- [] Uma hipótese de banco de dados para a EasyFood
- [] Uma justificativa para sua escolha
- [] Opcional: banco conectado e funcionando
- [] Se implementar o banco: ADR-002 registrando sua decisão

Próximo capítulo da EasyFood

Até agora:

```
GET /restaurants
|
ARRAY EM MEMÓRIA
|
POST /restaurants
```

Nossa próxima evolução será sair de:

```
API -> MEMÓRIA
```

para algo semelhante a:

```
API -> BANCO DE DADOS
```

Qual banco devemos escolher - e quais consequências essa escolha terá para a arquitetura da EasyFood?